

L43 — Collision Resolution: Chaining and Linear Probing

Unit: 3 | Chapter: Hashing | BT Level: BT5 | CO: CO2, CO3

Learning Objectives

- **Formulate** the collision inevitability principle using the Birthday Paradox and Pigeonhole Principle.
 - **Demystify the nomenclature inversion:** Master the confusing synonyms across literature (**Open Addressing** $\equiv$ **Closed Hashing** vs. **Separate Chaining** $\equiv$ **Open Hashing** $\equiv$ **Closed Addressing**).
 - **Implement and analyze** Closed Addressing / Separate Chaining, including singly linked lists, self-organizing chains, and treeified buckets (Java 8+ Red-Black trees).
 - **Implement, trace, and evaluate** Open Addressing probing techniques: Linear Probing, Quadratic Probing, and Double Hashing.
 - **Analyze clustering phenomena:** Contrast Primary Clustering (Linear Probing) with Secondary Clustering (Quadratic Probing), and prove how Double Hashing eliminates both.
 - **Resolve the Deletion Invariant Crisis:** Master Tombstones (Lazy Deletion) and Backward Shift Deletion.
 - **Explore modern advanced hashing:** Robin Hood Hashing, Cuckoo Hashing, Hopscotch Hashing, Coalesced Hashing, and Swiss Tables (SIMD control bytes).
 - **Evaluate empirical hardware tradeoffs:** Contrast CPU cache line locality against mathematical probe bounds.
-

1. The Collision Inevitability Principle

1.1 The Pigeonhole Principle

Let U be the universe of all possible keys (e.g., all valid 9-digit student IDs, $|U| = 10^9$) and let m be the number of available slots in the hash table array (e.g., $m = 10,000$). Because $|U| \gg m$, Dirichlet's **Pigeonhole Principle** guarantees that for any hash function $h: U \rightarrow \{0, 1, \dots, m-1\}$, there exist distinct keys $k_1, k_2 \in U$ ($k_1 \neq k_2$) such that:

$$h(k_1) = h(k_2)$$

A collision occurs whenever two distinct keys map to the exact same table slot.

1.2 The Birthday Paradox & Probabilistic Collisions

Even when the number of inserted keys n is vastly smaller than the table size m , collisions occur far earlier than human intuition suggests.

Under the assumption of uniform hashing (each key is independently and uniformly mapped to any of the m slots), the probability that n keys hash into distinct slots is:

$$P(\text{no collision}) = \prod_{k=1}^{n-1} \left(1 - \frac{k}{m}\right)$$

Using the Taylor series approximation $e^{-x} \approx 1 - x$ for small x :

$$P(\text{no collision}) \approx \prod_{k=1}^{n-1} e^{-k/m} = \exp\left(-\sum_{k=1}^{n-1} \frac{k}{m}\right) = \exp\left(-\frac{n(n-1)}{2m}\right) \approx e^{-n^2 / (2m)}$$

Therefore, the probability of **at least one collision** is:

$$P(\text{collision}) = 1 - P(\text{no collision}) \approx 1 - e^{-n^2 / (2m)}$$

Setting $P(\text{collision}) = 0.5$ yields $e^{-n^2 / (2m)} = 0.5 \implies \frac{n^2}{2m} = \ln 2$, leading to:

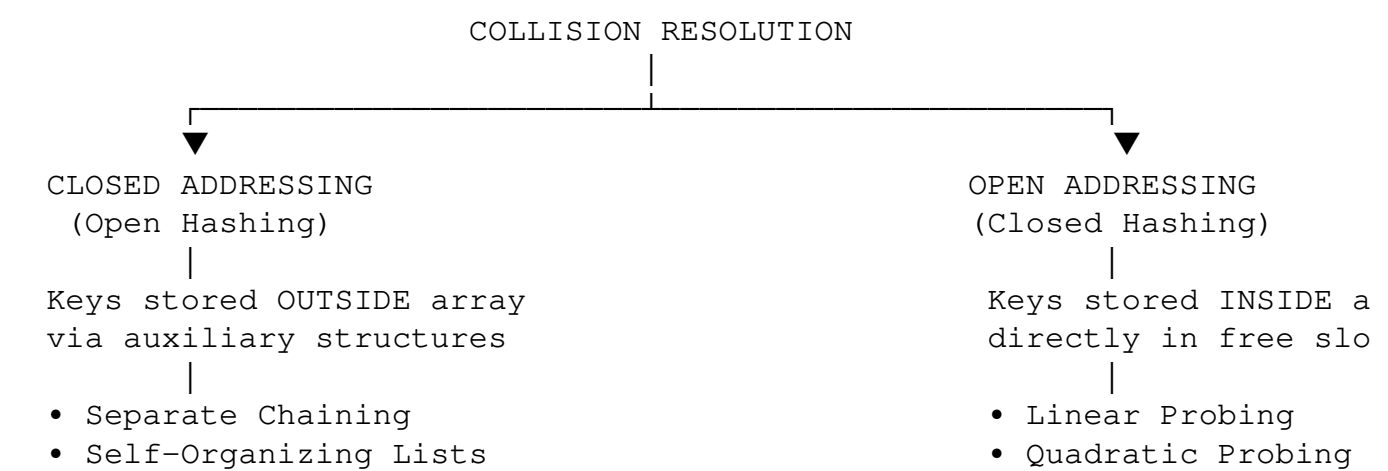
$$n \approx \sqrt{2m \ln 2} \approx 1.177 \sqrt{m}$$

Table Size (m)	Insertions (n) for $P(\text{collision}) \geq 50\%$	Practical Implication
$m = 365$ (Days in year)	$n = 23$ people	Classic Birthday Problem
$m = 1,000$ slots	$n = 38$ keys	Collisions begin almost immediately
$m = 10,000$ slots	$n = 119$ keys	Table is barely 1.2% full
$m = 1,000,000$ slots	$n = 1,178$ keys	Table is barely 0.12% full
$m = 2^{64}$ (64-bit hash values)	$n \approx 5.1 \times 10^9$ keys	Collisions guaranteed at $\approx 2^{32}$ elements

[!IMPORTANT] **Key Insight:** Collisions are not anomalies, bugs, or rare edge cases. They are an intrinsic mathematical certainty. A hash table's viability depends completely on the efficiency of its **collision resolution strategy**.

2. The Great Nomenclature Demystification

Computer science literature contains a notoriously confusing terminology inversion between the terms **Open** and **Closed**. Students and engineers frequently mix them up because "Open" and "Closed" mean opposite things depending on whether you are talking about **Addressing** or **Hashing**.



- Treeified Buckets
- Unrolled Linked Lists

- Double Hashing
- Robin Hood Hashing

2.1 The Two Fundamental Philosophies

1. Open Addressing = Closed Hashing

- **Why "Open Addressing"?** The *address* (slot index) at which key k is stored is **open to change**. If home slot $h(k)$ is occupied, the algorithm probes alternative addresses across the table until an empty slot is found. Key k may end up at any address in $\{0, 1, \dots, m-1\}$.
- **Why "Closed Hashing"?** All keys are stored strictly **closed within the fixed boundary** of the table array. No external storage, pointer nodes, or heap allocations exist outside the array bounds.

2. Separate Chaining = Open Hashing = Closed Addressing = Direct Chaining

- **Why "Open Hashing"?** The storage capacity is **open to expand dynamically** outside the table array into auxiliary memory (heap-allocated linked nodes, trees, or dynamic vectors).
- **Why "Closed Addressing"?** The *address* of key k is strictly **closed (fixed)** to bucket $h(k)$. Key k will *never* be stored at slot $h(k) + 1$ or $h(k) + 2$. It resides exclusively within the chain rooted at index $h(k)$.

2.2 The Complete Synonym Matrix

Primary Term	Equivalent Synonyms	Memory Model	Where Colliding Keys Reside	Load Factor Constraint	Key References
Open Addressing	Closed Hashing, Probing Hashing, Internal Hashing, Scatter Storage	Fixed contiguous array; zero dynamic pointer overhead	In other available slots <i>inside</i> the array itself	Strictly $\alpha < 1$ (typically $\alpha \leq 0.7$)	Knuth (TAOCP Vol 3), Peterson (1957), CLRS Ch. 11.4
Separate Chaining	Open Hashing, Closed Addressing, Direct Chaining, External Hashing	Array of bucket headers + dynamic heap nodes	<i>Outside</i> the array in linked lists or balanced trees	Unbounded α ($\alpha > 1$ allowed, typically $\alpha \leq 0.75$)	Luhn (1953), Knuth (TAOCP Vol 3), CLRS Ch. 11.2
Coalesced Hashing	Coalesced Chaining, Internal Chaining	Fixed table array containing both keys and next pointers	Inside the array itself, chained via internal index pointers	Strictly $\alpha \leq 1$	Williams (1959), Vitter & Chen (1987)

3. Strategy 1: Closed Addressing (Separate Chaining)

In **Separate Chaining**, each table slot acts as a bucket header pointing to an auxiliary data structure holding all records that hash to that specific slot.

Table Size $m = 7$

Keys inserted: 10, 22, 31, 4, 15, 28, 17, 88

Hash function: $h(k) = k \% 7$

Slot 0: [•] $\longrightarrow$ [28] $\longrightarrow$ None
Slot 1: [•] $\longrightarrow$ [15] $\longrightarrow$ [22] $\longrightarrow$ [88] $\longrightarrow$ None
Slot 2: [•] $\longrightarrow$ None (Empty)
Slot 3: [•] $\longrightarrow$ [17] $\longrightarrow$ [31] $\longrightarrow$ [10] $\longrightarrow$ None
Slot 4: [•] $\longrightarrow$ [4] $\longrightarrow$ None
Slot 5: [•] $\longrightarrow$ None (Empty)
Slot 6: [•] $\longrightarrow$ None (Empty)

3.1 Structural Variants of Chaining

1. Standard Singly Linked List (Head Insertion):

- Inserting a new key at the head takes $O(1)$ time unconditionally (assuming duplicates are allowed or verified).
- If duplicates must be overwritten, traversal of the chain is required ($O(1 + \alpha)$).

2. Self-Organizing Chains (Heuristic Search Optimization):

- **Move-To-Front (MTF)**: Whenever a search finds key k , splice its node and move it to the head of the chain. Highly effective for 80/20 skewed Pareto access patterns.
- **Transpose**: Swap the found node with its immediate predecessor.

3. Treeified Buckets (Java 8 + `HashMap` Technique):

- When any individual bucket exceeds `TREEIFY_THRESHOLD = 8` entries and total table capacity ≥ 64 , the linked list converts into a **Red-Black Tree**.
- Worst-case lookup drops from $O(n)$ to $O(\log n)$, neutralizing HashDoS (Denial of Service) attacks where malicious actors intentionally craft colliding keys.
- If deletions shrink the bucket count to ≤ 6 , it untreeifies back into a simple linked list.

4. Unrolled Linked Lists (Cache-Conscious Chaining):

- Each node contains a small fixed-capacity array (e.g., 4 or 8 slots) instead of a single element, amortizing pointer overhead and boosting CPU cache hit rates.

3.2 Formal Mathematical Analysis of Chaining

Let the load factor be $\alpha = \frac{n}{m}$, where n is the number of keys and m is the number of slots.

Theorem 1: Expected Unsuccessful Search Time

Under the **Simple Uniform Hashing Assumption (SUHA)** (any key is equally likely to hash

to any slot, independently of all other keys), an unsuccessful search takes expected time:

$$\mathbb{E}[\text{Time}] = \Theta(1 + \alpha)$$

Proof: Computing $h(k)$ takes $O(1)$. In an unsuccessful search, the entire chain at index $h(k)$ must be traversed to its tail. The expected number of elements in chain $T[h(k)]$ is:

$$\mathbb{E}[n_{h(k)}] = \sum_{j=1}^n \frac{1}{m} = \frac{n}{m} = \alpha$$

Including the initial $O(1)$ slot access, the total expected comparisons is $1 + \alpha$. $\square$

Theorem 2: Expected Successful Search Time

Under SUHA, a successful search takes expected time:

$$\mathbb{E}[\text{Time}] = \Theta\left(1 + \frac{\alpha}{2}\right)$$

Proof: Assume keys are inserted at the head of the list. When key x_i (the i -th key inserted) was added, the expected number of keys already in its chain was $\frac{i-1}{m}$. The number of elements examined during a search for x_i is 1 (for x_i itself) plus the keys inserted *after* x_i that landed in the same bucket. Averaging over all n keys:

$$\begin{aligned} \mathbb{E}[\text{comparisons}] &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m}\right) = 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{1}{nm} \\ &\quad \cdot \frac{n(n-1)}{2} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \end{aligned}$$

For large n , this simplifies to $1 + \frac{\alpha}{2}$. $\square$

3.3 Python Implementation: Robust Separate Chaining

```
"""
```

```
ChainedHashTable: Closed Addressing / Open Hashing implementation
featuring dynamic capacity resizing and full CRUD operations.
```

```
"""
```

```
from typing import Any, Optional, List, Tuple
```

```
class ChainedHashTable:
```

```
    def __init__(self, capacity: int = 11, max_load_factor: float = 0.7
        if capacity <= 0:
            raise ValueError("Capacity must be positive")
        self.capacity: int = capacity
        self.max_load_factor: float = max_load_factor
        self.size: int = 0
        self.table: List[List[Tuple[Any, Any]]] = [[] for _ in range(ca
```

```
    def _hash(self, key: Any) -> int:
        return hash(key) % self.capacity
```

```
    @property
```

```
    def load_factor(self) -> float:
        return self.size / self.capacity
```

```

def insert(self, key: Any, value: Any) -> None:
    """Insert or update a key-value pair. O(1) average time."""
    if self.load_factor >= self.max_load_factor:
        self._resize(self.capacity * 2 + 1)

    slot = self._hash(key)
    chain = self.table[slot]

    for i, (k, _) in enumerate(chain):
        if k == key:
            chain[i] = (key, value) # Update in place
            return

    chain.append((key, value))
    self.size += 1

def search(self, key: Any) -> Optional[Any]:
    """Retrieve value by key. O(1 + alpha) average time."""
    slot = self._hash(key)
    for k, v in self.table[slot]:
        if k == key:
            return v
    return None

def delete(self, key: Any) -> bool:
    """Remove key-value pair. Returns True if deleted, False otherwise"""
    slot = self._hash(key)
    chain = self.table[slot]
    for i, (k, _) in enumerate(chain):
        if k == key:
            chain.pop(i)
            self.size -= 1
            return True
    return False

def _resize(self, new_capacity: int) -> None:
    """Rehash all existing entries into a newly allocated table."""
    old_table = self.table
    self.capacity = new_capacity
    self.table = [[] for _ in range(new_capacity)]
    self.size = 0
    for chain in old_table:
        for k, v in chain:
            self.insert(k, v)

def __len__(self) -> int:
    return self.size

def __contains__(self, key: Any) -> bool:
    return self.search(key) is not None

```

4. Strategy 2: Open Addressing (Closed Hashing)

In **Open Addressing**, all keys reside directly inside the table array. No auxiliary linked structures exist. If an insertion collision occurs at home slot $h'(k)$, the algorithm computes a deterministic sequence of alternate slots called a **probe sequence**:

$$\langle h(k, 0), h(k, 1), h(k, 2), \dots, h(k, m-1) \rangle$$

General probing equation:

$$h(k, i) = (h'(k) + f(i)) \bmod m \quad \text{for probe index } i \in \{0, 1, \dots, m-1\}$$

Because all items occupy table cells directly, the table capacity m is a hard ceiling: $\alpha = \frac{n}{m} \leq 1$

4.1 Theoretical Upper Bounds Under Uniform Hashing

Assuming the **Uniform Hashing Assumption** (every key is equally likely to produce any of the $m!$ permutations of slots as its probe sequence):

- Unsuccessful Search / Insertion:** The probability that the first probe is occupied is $\frac{n}{m} = \alpha$. The probability that the first two probes are occupied is $\frac{n}{m} \cdot \frac{n-1}{m-1} < \alpha^2$. Expected probes: $E[\text{probes}] \leq 1 + \alpha + \alpha^2 + \alpha^3 + \dots = \frac{1}{1 - \alpha}$
- Successful Search:** Averaging over the cost of inserting each of the n keys: $E[\text{probes}] \leq \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right)$

Load Factor (α)	Unsuccessful Search Bound $\frac{1}{1-\alpha}$	Successful Search Bound $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$
$\alpha = 0.50$	\$2.00 probes	\$1.39 probes
$\alpha = 0.75$	\$4.00 probes	\$1.85 probes
$\alpha = 0.90$	\$10.00 probes	\$2.56 probes
$\alpha = 0.99$	\$100.00 probes	\$4.65 probes

5. Probing Techniques in Open Addressing

5.1 Linear Probing

Formulation

Offset function is linear: $f(i) = c \cdot i$ (standard case $c = 1$):

$$h(k, i) = (h'(k) + i) \bmod m$$

Number of Probe Sequences

There are **only m distinct probe sequences**. Once the initial slot $h'(k)$ is determined, the entire remaining probe path is completely fixed.

The Primary Clustering Phenomenon

Linear probing suffers severely from **Primary Clustering** (first-order clustering):

- Occupied slots form contiguous blocks (clusters).
- As clusters grow, the probability that a new random insertion hits the cluster increases proportional to the cluster's length!
- Specifically, an existing cluster of size L will grow if an incoming key hashes to any of the L occupied slots, **or** to the empty slot immediately following it. Hence, the probability of expanding the cluster is:

$$P(\text{grow cluster of size } L) = \frac{L + 1}{m}$$

This creates a vicious snowball effect: **large clusters attract more keys and grow even faster.**

Slots:	0	1	2	3	4	5	6	7	8	9
State:	[•]	[X]	[X]	[X]	[X]	[•]	[•]	[X]	[•]	[•]

└──────────┘
Cluster of 4

Probability of hitting cluster = $(4 + 1) / 10 = 50\%$

Knuth's Exact Derivation for Linear Probing (1962)

Because of primary clustering, linear probing performs worse than ideal uniform hashing. Donald Knuth derived the exact formulas:

$$\mathbb{E}[\text{Unsuccessful Search}] \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

$$\mathbb{E}[\text{Successful Search}] \approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

At $\alpha = 0.9$, an unsuccessful search under ideal uniform hashing takes 10 probes, but under linear probing it takes $\frac{1}{2}(1 + 100) \approx \mathbf{50.5}$ **probes!**

The Hardware Reality: Cache Locality

Despite mathematical clustering, linear probing is often the **fastest method in practice for $\alpha \leq 0.6$** :

- Modern CPU architectures load data into cache lines (typically 64 bytes).
 - Probing sequential array indices $i, i + 1, i + 2$ accesses memory already residing in the L1/L2 cache.
 - Chaining and non-linear probing suffer frequent CPU cache misses (fetching from RAM takes 50–200 cycles, dwarfing probe count math).
-

5.2 Quadratic Probing

Formulation

Offset function is quadratic: $f(i) = c_1 i + c_2 i^2$:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

Common configurations:

- $c_1 = 0, c_2 = 1 \implies h(k, i) = (h'(k) + i^2) \bmod m$
- $c_1 = 1, c_2 = 1 \implies h(k, i) = (h'(k) + i + i^2) \bmod m$
- $c_1 = 0.5, c_2 = 0.5 \implies h(k, i) = \left(h'(k) + \frac{i + i^2}{2}\right) \bmod m$

Elimination of Primary Clustering & Secondary Clustering

- Because the probe step jumps by $1, 4, 9, 16, 25, \dots$, contiguous clusters do not build up. Primary clustering is eliminated.
- **Secondary Clustering:** Keys that share the same initial home slot ($h'(k_1) = h'(k_2)$) will trace the **exact same quadratic probe sequence**. There are still only m distinct probe sequences across the entire table.

Full Table Coverage Theorem

A critical trap in quadratic probing is that it might loop through an orbit of already occupied slots and fail to find an empty slot, even when the table is not full.

[!TIP] **Theorem (Cormen, Leiserson, Rivest, Stein / Mark Allen Weiss):** If m is a prime number and the probing function is $h(k, i) = (h'(k) + i^2) \bmod m$, then for all $0 \leq i < j \leq \lfloor m/2 \rfloor$, the probe slots $h(k, i)$ and $h(k, j)$ are distinct.

Proof: Suppose $h'(k) + i^2 \equiv h'(k) + j^2 \pmod m$. Then: $i^2 \equiv j^2 \pmod m \implies i^2 - j^2 \equiv 0 \pmod m \implies (i - j)(i + j) \equiv 0 \pmod m$. Since m is prime, m must divide either $(i - j)$ or $(i + j)$. However, $0 \leq i < j \leq \lfloor m/2 \rfloor$, so:

- $0 < j - i < m/2 \implies m \nmid (j - i)$
- $0 < i + j < m \implies m \nmid (i + j)$

Neither term can be divisible by m . Contradiction! $\square$

Crucial Corollary: If m is prime, quadratic probing is **guaranteed to find an empty slot as long as the table is at least half empty** ($\alpha < 0.5$).

5.3 Double Hashing

Formulation

Offset function uses a secondary independent hash function $h_2(k)$ multiplied by i :

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Distinct Probe Sequences

Because the step size depends on key k itself via $h_2(k)$, the probe sequence depends on *both* $h_1(k)$ and $h_2(k)$. This yields $\Theta(m^2)$ distinct probe sequences, providing the closest practical approximation to the ideal Uniform Hashing Assumption and eliminating both primary and secondary clustering!

Strict Constraints on $h_2(k)$

1. **Non-Zero Condition:** $h_2(k) \neq 0$ for all k . (If $h_2(k) = 0$, the step size is zero and probing gets stuck at $h_1(k)$ in an infinite loop).
2. **Coprimality Condition:** $\gcd(h_2(k), m) = 1$. The step size must be relatively prime to table size m , ensuring that the probe sequence visits every single slot in $\{0, 1, \dots, m-1\}$ before repeating.

Common Parameter Designs

- **Prime Table Size:** Choose m as prime. $h_1(k) = k \bmod m$ $h_2(k) = 1 + (k \bmod (m - 1)) \bmod m$ or $h_2(k) = m - (k \bmod m) \bmod m$ ($\text{where } m' < m \text{ is prime}$)
- **Power of Two Table Size:** Choose $m = 2^p$. Ensure $h_2(k)$ always evaluates to an **odd integer**: $h_2(k) = 2 \cdot h'(k) + 1$ Every odd number is coprime to 2^p , guaranteeing full table traversal.

6. The Deletion Invariant Crisis in Open Addressing

6.1 Why Naive Deletion Breaks the Search Invariant

In open addressing, an unsuccessful search terminates the moment it encounters an empty (None) slot. Consider inserting k_1, k_2, k_3 where all three collide at slot 3 under linear probing:

- k_1 sits at slot 3
- k_2 probes and sits at slot 4
- k_3 probes past 3 and 4, and sits at slot 5

Initial State:

```
Index:   0     1     2     3     4     5     6
Table: [   ] [   ] [   ] [ k1 ] [ k2 ] [ k3 ] [   ]
```

Now, suppose we naively delete k_2 by setting `table[4] = None`:

After Naive Deletion of k_2 :

```
Index:   0     1     2     3     4     5     6
Table: [   ] [   ] [   ] [ k1 ] [None] [ k3 ] [   ]
```

Now, search for k_3 :

1. Check slot $h(k_3) = 3 \implies k_1 \neq k_3$.

2. Probe next slot \$4 \implies\$ **Encounters None!**
 3. Search algorithm assumes \$k_3\$ was never inserted and returns NOT FOUND! **\$k_3\$ has become an unreachable ghost element.** Naive deletion destroys search correctness.
-

6.2 Solution 1: Tombstones / Lazy Deletion (DELETED Sentinel)

Instead of clearing the slot to None, mark it with a sentinel object: DELETED (often called a **tombstone**).

State with Tombstone:

```
Index:    0      1      2      3          4          5      6
Table: [    ] [    ] [    ] [ k1 ] [DELETED] [ k3 ] [    ]
```

Operational Rules for Tombstones

- **Search(\$k\$):** Treat DELETED as an occupied mismatch. Continue probing! Stop only when key is found or genuine None is hit.
- **Insert(\$k\$):** Record the *first* DELETED slot encountered during probing. If key is not found in the rest of the chain, place the new key into that first recorded DELETED slot!
- **Delete(\$k\$):** Locate key, replace entry with DELETED.

Phantom Load Factor & Performance Degradation

Tombstones degrade search performance over time because unsuccessful searches must traverse past tombstones as if they were active keys. Define the effective load factor:

$\alpha_{\text{eff}} = \frac{n + t}{m} \quad (\text{where } t \text{ is the number of active tombstones})$

If α_{eff} approaches 1, searches degrade to $O(m)$ even if n is very small.
Remedy: Rehash/compact the table whenever $t > \frac{m}{4}$ or $\alpha_{\text{eff}} \geq 0.75$.

6.3 Solution 2: Backward Shift Deletion (Tombstone-Free)

For Linear Probing, we can completely eliminate tombstones by shifting subsequent elements backward into the vacated hole if their natural hash permits it.

Backward Shift Algorithm

```
def delete_backward_shift(table, keys, values, slot_to_delete, capacity):
    keys[slot_to_delete] = None
    values[slot_to_delete] = None
    hole = slot_to_delete
    curr = (slot_to_delete + 1) % capacity

    while keys[curr] is not None:
        home = hash(keys[curr]) % capacity
        # Check if shifting keys[curr] into 'hole' preserves cyclic ord
```

```
# Specifically, whether 'hole' lies cyclically between 'home' a
if (curr - home) % capacity >= (hole - home) % capacity:
    keys[hole] = keys[curr]
    values[hole] = values[curr]
    keys[curr] = None
    values[curr] = None
    hole = curr
curr = (curr + 1) % capacity
```

Backward shift deletion guarantees that the table remains entirely clean of tombstones, preserving optimal probe lengths indefinitely.

7. Concrete Step-by-Step Comparative Traces

Let table size be $m = 7$. Insert the following sequence of 7 keys: $\{\text{Keys: } [10, 22, 31, 4, 15, 28, 17]\}$

Base hash function: $h(k) = k \bmod 7$

- $h(10) = 10 \bmod 7 = \mathbf{3}$
- $h(22) = 22 \bmod 7 = \mathbf{1}$
- $h(31) = 31 \bmod 7 = \mathbf{3}$ (Collides with 10)
- $h(4) = 4 \bmod 7 = \mathbf{4}$
- $h(15) = 15 \bmod 7 = \mathbf{1}$ (Collides with 22)
- $h(28) = 28 \bmod 7 = \mathbf{0}$
- $h(17) = 17 \bmod 7 = \mathbf{3}$ (Collides with 10, 31)

7.1 Trace: Linear Probing

Probing formula: $h(k, i) = (h(k) + i) \bmod 7$

Key	Home $h(k)$	Probe Sequence Checked	Collisions	Final Slot	State of Table $[0, 1, 2, 3, 4, 5, 6]$
10	3	[3]	0	3	[_, _, _, 10, _, _]
22	1	[1]	0	1	[_, 22, _, 10, _, _]
31	3	[3:10 occupied] $\rightarrow$ [4:empty]	1	4	[_, 22, _, 10, 31, _]
4	4	[4:31 occupied] $\rightarrow$ [5:empty]	1	5	[_, 22, _, 10, 31, 4]
15	1	[1:22 occupied] $\rightarrow$ [2:empty]	1	2	[_, 22, 15, 10, 31, 4]
28	0	[0]	0	0	[28, 22, 15, 10, 31, 4]
17	3	[3:10] $\rightarrow$ [4:31] $\rightarrow$ [5:4] $\rightarrow$ [6:empty]	3	6	[28, 22, 15, 10, 31, 4, 17]

[!WARNING] **Observation on Primary Clustering:** Notice key 17: even though it only collided with 10 initially, it was forced to traverse slots 4 and 5 because previous insertions (31 and 4) formed a contiguous block across slots 1 to 5.

7.2 Trace: Quadratic Probing

Probing formula: $h(k, i) = (h(k) + i + i^2) \bmod 7$ Offsets for $i = 0, 1, 2, 3$:

- $i = 0 \implies +0$
- $i = 1 \implies 1 + 1 = +2$
- $i = 2 \implies 2 + 4 = +6$
- $i = 3 \implies 3 + 9 = 12 \equiv +5$

Key	Home $h(k)$	Probe Calculations	Collisions	Final Slot	State of Table $[0, 1, 2, 3, 4, 5, 6]$
10	3	$i=0: 3$	0	3	[_, _, _, 10, _, _, _]
22	1	$i=0: 1$	0	1	[_, 22, _, 10, _, _, _]
31	3	$i=0: 3$ (10!) $\rightarrow i=1: (3+2)\%7 = \mathbf{5}$	1	5	[_, 22, _, 10, _, 31, _]
4	4	$i=0: 4$	0	4	[_, 22, _, 10, 4, 31, _]
15	1	$i=0: 1$ (22!) $\rightarrow i=1: (1+2)\%7 = \mathbf{3}$ (10!) $\rightarrow i=2: (1+6)\%7 = \mathbf{0}$	2	0	[15, 22, _, 10, 4, 31, _]
28	0	$i=0: 0$ (15!) $\rightarrow i=1: (0+2)\%7 = \mathbf{2}$	1	2	[15, 22, 28, 10, 4, 31, _]
17	3	$i=0: 3$ (10!) $\rightarrow i=1: (3+2)\%7 = 5$ (31!) $\rightarrow i=2: (3+6)\%7 = \mathbf{2}$ (28!) $\rightarrow i=3: (3+5)\%7 = \mathbf{1}$ (22!) $\dots$	4+	Cyclic orbit	Primary cluster avoided, but orbit hit occupied slots

7.3 Trace: Double Hashing

Formulas:

- $h_1(k) = k \bmod 7$
- $h_2(k) = 1 + (k \bmod 5)$ (Guarantees step size $\in \{1, 2, 3, 4, 5\}$, coprime to 7)

Step sizes:

- Key 10: $h_2(10) = 1 + (10 \bmod 5) = \mathbf{1}$
- Key 22: $h_2(22) = 1 + (22 \bmod 5) = \mathbf{3}$
- Key 31: $h_2(31) = 1 + (31 \bmod 5) = \mathbf{2}$
- Key 4: $h_2(4) = 1 + (4 \bmod 5) = \mathbf{5}$
- Key 15: $h_2(15) = 1 + (15 \bmod 5) = \mathbf{1}$

- Key 28: $h_2(28) = 1 + (28 \bmod 5) = \mathbf{4}$
- Key 17: $h_2(17) = 1 + (17 \bmod 5) = \mathbf{3}$

Key	$h_1(k)$	Step $h_2(k)$	Probe Sequences Computed	Collisions	Final Slot	State of Table \$[0, 1, 2, 3, 4, 5, 6]\$
10	3	1	$i=0: \mathbf{3}$	0	3	[_, _, _, 10, _, _, _]
22	1	3	$i=0: \mathbf{1}$	0	1	[_, 22, _, 10, _, _, _]
31	3	2	$i=0: 3 \text{ (10!)} \rightarrow i=1: (3 + 1 \cdot 2) \% 7 = \mathbf{5}$	1	5	[_, 22, _, 10, _, 31, _]
4	4	5	$i=0: \mathbf{4}$	0	4	[_, 22, _, 10, 4, 31, _]
15	1	1	$i=0: 1 \text{ (22!)} \rightarrow i=1: (1 + 1 \cdot 1) \% 7 = \mathbf{2}$	1	2	[_, 22, 15, 10, 4, 31, _]
28	0	4	$i=0: \mathbf{0}$	0	0	[28, 22, 15, 10, 4, 31, _]
17	3	3	$i=0: 3 \text{ (10!)} \rightarrow i=1: (3 + 1 \cdot 3) \% 7 = \mathbf{6}$	1	6	[28, 22, 15, 10, 4, 31, 17]

[!TIP] **Key Takeaway:** Double hashing placed key 17 into slot 6 after **only 1 collision**, compared to **3 collisions** under Linear Probing! Double hashing completely shattered the cluster.

8. Advanced & Modern Collision Resolution Strategies

Beyond the classical textbook methods, high-performance systems employ cutting-edge variants:

8.1 Robin Hood Hashing

Named after the folk hero who "took from the rich and gave to the poor":

- **Core Concept:** Every resident key tracks its **Probe Sequence Length (PSL)** — how far it currently sits from its home slot ($h'(k)$).
- **Insertion Rule:** When inserting key k with probe length p , probe slot by slot. If a slot is occupied by resident key r with probe length p_r :
 - If $p > p_r$: The incoming key is "poorer" (has traveled further) than the resident key. **Swap them!** Place k into the slot, and continue probing to insert the evicted key r (with probe length $p_r + 1$).
 - If $p \leq p_r$: Keep probing with key k .

- **Major Advantage:** Dramatically tightens the **variance** of search times.
- **Fast Search Abort:** During an unsuccessful search, if you encounter an occupied slot where resident $p_r < \text{current search probe } p$, the search can **terminate immediately with failure** (the key cannot exist past this point).

8.2 Cuckoo Hashing (Pagh & Rodler, 2001)

- Uses **two independent hash functions** $h_1(k)$ and $h_2(k)$.
- Key k is guaranteed to reside at either $table[h_1(k)]$ or $table[h_2(k)]$.
- **Worst-Case $O(1)$ Search and Deletion:** Always examines at most 2 slots!
- **Insertion:** Always attempt to place at $h_1(k)$. If occupied by x , evict x and move x to its alternate slot (like a cuckoo pushing other birds out of the nest). If x 's alternate slot is occupied, recursively evict that tenant. If an eviction cycle occurs, rehash the entire table with fresh hash functions.

8.3 Hopscotch Hashing (Herlihy, Shavit & Tzafrir, 2008)

- Blends Linear Probing with bounded neighborhood bitmasks.
- Guarantees that any key k is stored within a fixed neighborhood of H slots (e.g., $H = 32$) starting from $h(k)$.
- If insertion finds a free slot beyond H , it executes a sequence of swaps to displace existing items and pull the free slot closer into the neighborhood.
- Extremely cache-friendly and well-suited for high-concurrency multi-threaded lock-free tables.

8.4 Coalesced Hashing (Vitter & Chen, 1987)

- Combines chaining and open addressing: keys reside strictly in the table array, but each slot includes a `next` pointer pointing to another slot in the same array.
- Avoids dynamic memory overhead while retaining linked-list traversal efficiency.

8.5 Swiss Tables (Google Abseil / Rust `hashbrown`)

- State-of-the-art hash table in modern systems.
- Splits each 64-bit hash into:
 - **H1 (top 57 bits):** Selects a group of 16 slots.
 - **H2 (bottom 7 bits):** Acts as a 1-byte fingerprint.
- Maintains a contiguous control byte array where each byte stores an H2 fingerprint or status (`EMPTY`, `DELETED`).
- Uses **CPU SIMD vector instructions** (`_mm_cmpeq_epi8`) to compare 16 fingerprints simultaneously in a single CPU clock cycle!

9. Python 3 Implementations: Open Addressing

9.1 Linear Probing with Tombstones & Probe Monitoring

"""

LinearProbingHashTable: Open Addressing / Closed Hashing

with Tombstone Lazy Deletion and dynamic load-factor management.

"""

from typing import Any, Optional, List

class LinearProbingHashTable:

DELETED = object() # Unique sentinel object for deleted slots

def __init__(self, capacity: int = 11, max_load_factor: float = 0.6):
 self.capacity: int = capacity
 self.max_load_factor: float = max_load_factor
 self.size: int = 0
 self.tombstones: int = 0
 self.keys: List[Any] = [None] * capacity
 self.values: List[Any] = [None] * capacity

def _hash(self, key: Any) -> int:
 return hash(key) % self.capacity

@property

def load_factor(self) -> float:
 return self.size / self.capacity

@property

def effective_load_factor(self) -> float:
 return (self.size + self.tombstones) / self.capacity

def insert(self, key: Any, value: Any) -> int:

"""

Insert or update (key, value).
 Returns the number of probes required.

"""

if self.effective_load_factor >= self.max_load_factor:
 self._resize(self.capacity * 2 + 1)

slot = self._hash(key)
 first_tombstone_idx: Optional[int] = None
 probes = 0

for i in range(self.capacity):
 probes += 1
 idx = (slot + i) % self.capacity
 curr_key = self.keys[idx]

if curr_key is None:

Key not found; insert into tombstone if found, else e
 target = first_tombstone_idx if first_tombstone_idx is
 if first_tombstone_idx is not None:
 self.tombstones -= 1
 self.keys[target] = key
 self.values[target] = value
 self.size += 1

```

        return probes

    elif curr_key is self.DELETED:
        if first_tombstone_idx is None:
            first_tombstone_idx = idx

    elif curr_key == key:
        self.values[idx] = value # Update existing key
        return probes

# If loop exhausts but tombstone exists
if first_tombstone_idx is not None:
    self.keys[first_tombstone_idx] = key
    self.values[first_tombstone_idx] = value
    self.size += 1
    self.tombstones -= 1
    return probes

raise OverflowError("Hash table capacity exhausted")

def search(self, key: Any) -> Tuple[Optional[Any], int]:
    """
    Lookup value by key.
    Returns (value, probe_count).
    """
    slot = self._hash(key)
    probes = 0

    for i in range(self.capacity):
        probes += 1
        idx = (slot + i) % self.capacity
        curr_key = self.keys[idx]

        if curr_key is None:
            return (None, probes) # Unsuccessful search terminates
        if curr_key is not self.DELETED and curr_key == key:
            return (self.values[idx], probes)

    return (None, probes)

def delete(self, key: Any) -> bool:
    """Mark slot with DELETED sentinel."""
    slot = self._hash(key)
    for i in range(self.capacity):
        idx = (slot + i) % self.capacity
        curr_key = self.keys[idx]
        if curr_key is None:
            return False
        if curr_key is not self.DELETED and curr_key == key:
            self.keys[idx] = self.DELETED
            self.values[idx] = None
            self.size -= 1

```

```

        self.tombstones += 1
        return True
    return False

def _resize(self, new_capacity: int) -> None:
    old_keys = self.keys
    old_values = self.values

    self.capacity = new_capacity
    self.keys = [None] * new_capacity
    self.values = [None] * new_capacity
    self.size = 0
    self.tombstones = 0

    for k, v in zip(old_keys, old_values):
        if k is not None and k is not self.DELETED:
            self.insert(k, v)

```

9.2 Double Hashing Implementation

"""

DoubleHashingHashTable: Complete implementation of double hashing with guaranteed non-zero and coprime secondary step size.

"""

from typing import Any, Optional, Tuple, List

class DoubleHashingHashTable:

DELETED = object()

```

    def __init__(self, capacity: int = 11, max_load_factor: float = 0.6):
        self.capacity = capacity
        self.max_load_factor = max_load_factor
        self.size = 0
        self.tombstones = 0
        self.keys: List[Any] = [None] * capacity
        self.values: List[Any] = [None] * capacity

```

```

    def _hash1(self, key: Any) -> int:
        return hash(key) % self.capacity

```

```

    def _hash2(self, key: Any) -> int:
        # Step size must be non-zero and coprime to self.capacity.
        # For prime capacity m, 1 + (hash(key) % (m - 1)) is strictly i
        prime_sub = max(1, self.capacity - 1)
        return 1 + (abs(hash(key)) % prime_sub)

```

```

    def insert(self, key: Any, value: Any) -> int:
        if (self.size + self.tombstones) / self.capacity >= self.max_lo
            self._resize(self._next_prime(self.capacity * 2))

```

```

h1 = self._hash1(key)
step = self._hash2(key)
first_tombstone: Optional[int] = None
probes = 0

for i in range(self.capacity):
    probes += 1
    idx = (h1 + i * step) % self.capacity
    curr = self.keys[idx]

    if curr is None:
        target = first_tombstone if first_tombstone is not None
        if first_tombstone is not None:
            self.tombstones -= 1
        self.keys[target] = key
        self.values[target] = value
        self.size += 1
        return probes

    elif curr is self.DELETED:
        if first_tombstone is None:
            first_tombstone = idx

    elif curr == key:
        self.values[idx] = value
        return probes

if first_tombstone is not None:
    self.keys[first_tombstone] = key
    self.values[first_tombstone] = value
    self.size += 1
    self.tombstones -= 1
    return probes

raise OverflowError("Double hashing probe sequence exhausted")

def search(self, key: Any) -> Tuple[Optional[Any], int]:
    h1 = self._hash1(key)
    step = self._hash2(key)
    probes = 0

    for i in range(self.capacity):
        probes += 1
        idx = (h1 + i * step) % self.capacity
        curr = self.keys[idx]

        if curr is None:
            return (None, probes)
        if curr is not self.DELETED and curr == key:
            return (self.values[idx], probes)

    return (None, probes)

```

```

def _next_prime(self, n: int) -> int:
    while True:
        n += 1
        if self._is_prime(n):
            return n

    @staticmethod
    def _is_prime(n: int) -> bool:
        if n < 2:
            return False
        if n in (2, 3):
            return True
        if n % 2 == 0 or n % 3 == 0:
            return False
        i = 5
        while i * i <= n:
            if n % i == 0 or n % (i + 2) == 0:
                return False
            i += 6
        return True

    def _resize(self, new_capacity: int) -> None:
        old_keys = self.keys
        old_values = self.values
        self.capacity = new_capacity
        self.keys = [None] * new_capacity
        self.values = [None] * new_capacity
        self.size = 0
        self.tombstones = 0
        for k, v in zip(old_keys, old_values):
            if k is not None and k is not self.DELETED:
                self.insert(k, v)

```

10. Grand Comparative Tradeoff Matrix

Feature / Metric	Separate Chaining (Open Hashing)	Linear Probing (Closed Hashing)	Quadratic Probing	Double Hashing	Robin Hood Hashing
Storage Model	External linked nodes	Contiguous array	Contiguous array	Contiguous array	Contiguous array
Load Factor Bound	α unbounded ($\alpha > 1$ allowed)	$\alpha < 1$ (opt. $\alpha \leq 0.65$)	$\alpha < 0.5$ (prime m)	$\alpha < 1$ (opt. $\alpha \leq 0.7$)	$\alpha < 0.9$
Primary Clustering	None	Severe	None	None	None

Secondary Clustering	None	Inherent	Yes	None	None
Distinct Probe Paths	m buckets	m sequences	m sequences	$\Theta(m^2)$ sequences	m sequences
Avg. Unsuccessful Search	$1 + \frac{1}{\alpha}$	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$\approx \frac{1}{1-\alpha}$	$\frac{1}{1-\alpha}$	Fast early abort
Avg. Successful Search	$1 + \frac{1}{2\alpha}$	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$\approx \frac{1}{\alpha \ln \frac{1}{1-\alpha}}$	$\frac{1}{\alpha \ln \frac{1}{1-\alpha}}$	Extremely low variance
Cache Performance	Poor (pointer chasing)	Exceptional (L1/L2 prefetch)	Moderate	Poor (strided misses)	Excellent
Deletion Cost	Direct pointer unlink	Tombstone or backward shift	Tombstone required	Tombstone required	Backward shift / PSL shift
Memory Overhead	Pointer per node (\$8-16B)	Zero pointer overhead	Zero pointer overhead	Zero pointer overhead	1 byte PSL per slot

11. Real-World Language & Runtime Implementations

How major programming languages resolve collisions in their standard libraries:

1. Python (`dict` & `set`)

- **Scheme:** Open Addressing.
- **Implementation:** Since Python 3.6, `dict` uses a compact table: a dense array of `(hash, key, value)` entries plus a sparse hash index table.
- **Probe Function:** Uses a pseudo-random perturbation recurrence to defeat clustering:

```
perturb = hash(key)
probe = (5 * probe + 1 + perturb) & mask
perturb >>= 5
```

2. Java (`java.util.HashMap` & `HashSet`)

- **Scheme:** Closed Addressing (Separate Chaining).
- **Implementation:** Array of buckets. Each bucket starts as a singly linked list (`Node<K, V>`).
- **Treeification:** If any bucket exceeds 8 entries (`TREEIFY_THRESHOLD`) and capacity ≥ 64 , the bucket morphs into a Red-Black Tree (`TreeNode<K, V>`), guaranteeing $O(\log n)$ worst-case lookup.

3. C++ (`std::unordered_map`)

- **Scheme:** Closed Addressing (Separate Chaining).

- **Implementation:** The ISO C++ standard mandates iterator validity guarantees (insertions must never invalidate pointers or references to unchanged elements), effectively requiring separate chaining with individual node allocations.

4. Rust (`std::collections::HashMap`)

- **Scheme:** Open Addressing via **Swiss Tables** (`hashbrown` crate).
- **Implementation:** SIMD-accelerated control byte metadata matching with 87.5% load factor ceiling.

5. Go (`map`)

- **Scheme:** Bucketed Hybrid Chaining.
- **Implementation:** Array of buckets where each bucket holds up to 8 key-value pairs inline, followed by overflow bucket pointers if collisions exceed 8 entries.

12. Common Exam & Interview Gotchas

[!CAUTION] **Trap 1: The Zero Step Size in Double Hashing Question:** *Why must $h_2(k)$ never evaluate to 0?* **Answer:** If $h_2(k) = 0$, the probing formula $h(k, i) = (h_1(k) + i \cdot 0) \bmod m = h_1(k)$ remains constant for all i . The algorithm enters an **infinite loop** checking the same occupied slot over and over!

[!NOTE] **Trap 2: Why Naive Deletion Destroys Open Addressing Question:** *Can we simply set `table[slot] = None` when deleting in linear probing?* **Answer: No.** Setting a slot to `None` creates an artificial barrier that terminates subsequent searches prematurely, making colliding elements inserted further down the probe chain completely unreachable. You must use either **tombstones** (`DELETED`) or **backward shift deletion**.

[!TIP] **Trap 3: Theory vs. Hardware Reality Question:** *Why do production systems often choose Linear Probing or Swiss Tables over Double Hashing, despite Double Hashing having superior mathematical probe bounds?* **Answer: CPU Cache lines.** Modern processors fetch 64-byte chunks from RAM. Linear probing searches adjacent slots that already sit in the L1/L2 cache. Double hashing generates pseudo-random strides across the array, causing an expensive CPU cache miss (~100–200 cycles) on nearly every single probe!

Summary

- **Collision Inevitability:** By the Birthday Paradox, collisions occur with $> 50\%$ probability after only $O(\sqrt{m})$ insertions.
- **The Core Synonyms:**
 - **Open Addressing** $\equiv$ **Closed Hashing:** Keys stored inside the array; address is open to probe; $\alpha < 1$.
 - **Separate Chaining** $\equiv$ **Open Hashing** $\equiv$ **Closed Addressing:** Keys stored outside the array in linked structures; address is closed/fixed; α unbounded.

- **Linear Probing:** Suffers from Primary Clustering ($\mathbb{E}[U] \approx \frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$), but excels in CPU cache locality.
 - **Quadratic Probing:** Eliminates primary clustering; exhibits secondary clustering; guarantees finding empty slot if m is prime and $\alpha < 0.5$.
 - **Double Hashing:** Eliminates both primary and secondary clustering using two coprime hash functions, generating $\Theta(m^2)$ probe sequences.
 - **Deletion in Open Addressing:** Requires tombstones (DELETED) or backward shift deletion to preserve search continuity.
 - **Modern Paradigms:** Robin Hood Hashing minimizes probe variance; Cuckoo Hashing guarantees $O(1)$ worst-case lookup; Swiss Tables leverage SIMD parallelism.
-

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.), Chapter 11: Hash Tables. MIT Press.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.), Section 6.4: Hashing. Addison-Wesley.
- Skiena, S. S. (2020). *The Algorithm Design Manual* (3rd ed.), Section 3.7: Hashing and Collision Resolution. Springer.
- Pagh, R., & Rodler, F. F. (2004). *Cuckoo Hashing*. *Journal of Algorithms*, 51(2), 122–144.
- Herlihy, M., Shavit, N., & Tzafrir, M. (2008). *Hopscotch Hashing*. DISC 2008, Lecture Notes in Computer Science, vol 5218. Springer.